Hızlı Başvuru Kağıdı #6: Çarpışma Algılama ve Fizik Temelleri

PyGame ile Oyun Geliştirme – Hafta 6

Rect Sınıfı

Rect Oluşturma

Yöntem	Açıklama
`Rect(x,y,w,h)`	Doğrudan oluştur
`Surface.get_rect()`	Surface'ten al
`get_rect(center=..)`	Konumlu oluştur

Rect Özellikleri

Özellik	Açıklama
`x, y`	Sol üst köşe
`width, height`	Boyutlar
`center, centerx, centery`	Merkez noktası
`top, bottom, left, right`	Kenar değerleri
`topleft, topright`	Köşe çiftleri
`midtop, midbottom`	Kenar orta noktalar
`size`	(width, height)

Rect Kullanımı

```
# Dogrudan olusturma
kutu = pygame.Rect(100, 200, 50, 50)

# Surface'ten alma
gorsel = pygame.image.load(
    "gemi.png").convert_alpha()
rect = gorsel.get_rect(center=(400,300))

# Ozelliklere erisim
print(rect.center)    # (400, 300)
print(rect.topleft)   # (375, 275)
print(rect.size)      # (50, 50)
```

Rect Metodları

Metot	Açıklama
`move(x, y)`	Yeni Rect döndürür
`move_ip(x, y)`	Yerinde kaydırma
`inflate(w, h)`	Boyutu değiştir (yeni)
`inflate_ip(w, h)`	Yerinde boyutlandır
`clamp(rect)`	Sınır içerisine al
`clamp_ip(rect)`	Yerinde sınırla
`union(rect)`	Birleştir
`clip(rect)`	Kesişim Rect'i

Rect Çarpışması

– Rect vs Rect

İki dikdörtgenin çakışıp çakışmadığını kontrol eder.

Dönüş değeri: `True` (çakışıyor) / `False`

`colliderect()` Kullanımı

```
oyuncu = pygame.Rect(100, 100, 50, 50)
dusman = pygame.Rect(130, 120, 40, 40)

if oyuncu.colliderect(dusman):
    print("Carpishma var!")
    can -= 1
```

– Rect vs Nokta

Bir noktanın (x, y) dikdörtgen içerisinde olup olmadığını kontrol eder. Fare tıklamaları için idealdir.

`collidepoint()` ile Fare Tıklama

```
buton = pygame.Rect(300, 200, 150, 50)

for olay in pygame.event.get():
    if olay.type == pygame.MOUSEBUTTONDOWN:
        if buton.collidepoint(olay.pos):
            print("Butona tiklandi!")
```

Diğer Çarpışma Metodları

Metot	Açıklama
`collidelist(list)`	Listedeki ilk çarpışan indeks
`collidelistall(list)`	Tüm çarpışan indeksler
`contains(rect)`	Tamamen içeriyor mu?

Sprite Çarpışma Fonksiyonları

– Sprite vs Grup

Bir sprite'ın bir gruptaki hangi sprite'larla çarpıştığını bulur.

```
pygame.sprite.spritecollide(sprite, group, dokill)
```

`dokill=True` → çarpışan sprite'lar gruptan silinir

`dokill=False` → sadece liste döner

`spritecollide()` Kullanımı

```
# Oyuncu ile carpisan dusmanlar
vurulanlar = pygame.sprite.spritecollide(
    oyuncu, dusman_grup, True)

for dusman in vurulanlar:
    puan += 10
    patlama_efekti(dusman.rect.center)
```

– Grup vs Grup

İki gruptaki sprite'ların birbirleriyle çarpışmasını kontrol eder.

```
pygame.sprite.groupcollide(
group1, group2, dokill1, dokill2)
```

Dönüş: `{sprite1: [sprite2, ...], ...}` sözlüğü

`groupcollide()` Kullanımı

```
# Mermiler vs dusmanlar
işaretler = pygame.sprite.groupcollide(
    mermi_grup, dusman_grup, True, True)

for mermi, dusmanlar in işaretler.items():
    for d in dusmanlar:
        puan += 50
```

Çarpışma Yöntemleri

Yöntem	Açıklama
Rect (varsayılan)	Dikdörtgen kutu, en hızlı
`collide_circle`	Daire tabanlı, orta hız
`collide_mask`	Piksel tabanlı, en doğru

`collide_mask` Kullanımı

```
# Sprite'a mask ekle
class Oyuncu(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.image.load(
            "gemi.png").convert_alpha()
        self.rect = self.image.get_rect()
        self.mask = pygame.mask.from_surface(
            self.image)

# Mask ile carpishma kontrolu
vurulanlar = pygame.sprite.spritecollide(
    oyuncu, dusmanlar, False,
    pygame.sprite.collide_mask)
```

Mask Performans Uyarısı

`collide_mask` en doğru sonucu verir ama en yavaş yöntemdir.

Strateji: Önce `colliderect()` ile ön eleme yap, sonra `collide_mask` ile hassas kontrol uygula.

collide_circle Kullanımı

Daire çarpışması için sprite'a `radius` özelliği ekle:
`self.radius = 20`
Eklenmezse otomatik hesaplanır (`image` boyutundan).

Temel Fizik Simülasyonu

Yerçekimi

Yerçekimi = Her karede hıza sabit değer ekleme

`hiz_y += YERCEKIMI`

`rect.y += hiz_y`

Tipik `YERCEKIMI` değeri: `0.3 – 1.0`

Yerçekimi Uygulaması

```
YERCEKIMI = 0.5
ZEMIN_Y = 550

class Top(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((30, 30),
            pygame.SRCALPHA)
        pygame.draw.circle(self.image,
            (255,100,0), (15,15), 15)
        self.rect = self.image.get_rect(
            center=(x, y))
        self.hiz_y = 0

    def update(self):
        self.hiz_y += YERCEKIMI
        self.rect.y += self.hiz_y
        if self.rect.bottom >= ZEMIN_Y:
            self.rect.bottom = ZEMIN_Y
            self.hiz_y = 0
```

Momentum ve Sürtünme

Kavram	Formül
Hız	`konum += hiz`
İvme	`hiz += ivme`
Yerçekimi	`hiz_y += g`
Sürtünme	`hiz_x *= 0.95`
Geri Sekme	`hiz_y *= -sekme`

Sürtünme ve Geri Sekme

```
SURTUNME = 0.95
SEKME_KATSAYISI = 0.7

def update(self):
    # Yercekimi
    self.hiz_y += YERCEKIMI
    # Surtunme (yatay)
    self.hiz_x *= SURTUNME
    # Konum güncelle
    self.rect.x += self.hiz_x
    self.rect.y += self.hiz_y
    # Zemin carpismasi + geri sekme
    if self.rect.bottom >= ZEMIN_Y:
        self.rect.bottom = ZEMIN_Y
        self.hiz_y *= -SEKME_KATSAYISI
        # Kucuk sektirmeleri durdur
        if abs(self.hiz_y) < 1:
            self.hiz_y = 0
```

Hasar Sistemi ve Yok Etme

Hasar Sistemi

```
class Dusman(pygame.sprite.Sprite):
    def __init__(self, x, y, can=3):
        super().__init__()
        self.image = pygame.Surface((40,40))
        self.image.fill((200, 30, 30))
        self.rect = self.image.get_rect(
            center=(x, y))
        self.can = can

    def hasar_al(self, miktar=1):
        self.can -= miktar
        if self.can <= 0:
            self.kill()  # Tum gruplardan sil
            return True  # Yok edildi
        return False
```

Breakout Mantığı

Eleman	Davranış
Raket	Fare/klavye ile yatay hareket
Top	`hiz_x`, `hiz_y` ile hareket
Tuğlalar	Çarpışmada hasar al / yok ol
Duvarlar	`hiz_x *= -1` (yan), top reset
Tavan	`hiz_y *= -1`

Top Geri Sekme (Breakout)

```
def update(self):
    self.rect.x += self.hiz_x
    self.rect.y += self.hiz_y

    # Yan duvarlar
    if self.rect.left <= 0 or \
       self.rect.right >= GENISLIK:
        self.hiz_x *= -1
    # Tavan
    if self.rect.top <= 0:
        self.hiz_y *= -1

    # Raket carpismasi
    if self.rect.colliderect(raket.rect):
        self.hiz_y *= -1
        self.rect.bottom = raket.rect.top
```

Sık Yapılan Hatalar

Hatalar ve Çözümler

Hata	Çözüm
`mask` tanımlamama	`collide_mask` için zorunlu
Tünelden geçme	Küçük adımlarla kontrol et
Sonsuz sekme	`abs(hiz) < 1` ise durdur
`dokill` yanlış	`True`: siler, `False`: silmez
Çarpışmada takılma	Konum düzelt, sonra yönü çevir
Fizik dt'siz	`dt = clock.tick(60)/1000`

Fizik Motoru Şablonu

Fizik döngüsü sırası:
1. Kuvvetleri uygula (yerçekimi, sürtünme)
2. Hızı güncelle (`hiz += ivme`)
3. Konumu güncelle (`konum += hiz`)
4. Çarpışma kontrolü yap
5. Çarpışma tepkisi uygula (sekme, hasar)